

DSA Problem Ladder — Day 4

Thursday — Medium: transfer and repair

September 17, 2026 | Priyanka | 120 minutes | C11 or Python 3

Central question: can the same reasoning survive a changed output or story?

1. Daily capacity checker. Given ordered positive parcel weights and a proposed capacity C , return the minimum number of contiguous shipping days needed, or -1 if any parcel exceeds C . Require $O(n)$ time and $O(1)$ space. Justify why a locally full day cannot increase the minimum number of days. This deliberately isolates Wednesday's difficult subproblem.
2. Smallest processing speed. Positive integer pile sizes must be processed in at most H hours, with H at least the number of piles. At speed k , a pile takes $\text{ceil}(\text{size}/k)$ hours; unfinished hours cannot be shared between piles. Find the smallest positive integer k . Require $O(n \log(M+1))$ time, M being the largest pile. Example: `[3, 6, 7, 11]`, $H=8$ gives 4. Explain how this feasibility test differs from shipping.
3. Closest pair sum. Given at least two unsorted integers and target T , return a pair of values at distinct indices minimizing the absolute difference between their sum and T . Any optimal pair is accepted. Require $O(n \log n)$ time; sorting a copy is allowed. Explain why your pointer movement cannot miss a better pair. Include duplicates and targets outside the possible sum range.

Session rules

- Minutes 0–10: closed-notes recall of one invariant and an edge case from the previous session (Monday: last week's boundary search).
- Minutes 10–110: solve today's queue in order. Check time after roughly 25 minutes per problem on easy days, 33 on medium days, and 50 on Wednesday. Unused time transfers forward; the queue is not a quota.
- Minutes 110–120: stop coding, record results, and explain one failed assumption and its correction. Mark unfinished work partial at the cutoff.
- Use C11 OR Python 3 consistently throughout the timed week. Translation is optional and does not count as another solve.
- Before coding, state a direct approach, its cost, and what property might remove work. After coding, give a correctness argument, time/space costs, and normal, boundary, and adversarial tests.
- After 15 minutes without a useful new step, write the blocker and choose to continue, request a small hint, or park the problem. Record hints; keep complete solutions closed during the attempt.

Count a solve only when working code meets the stated constraints, passes checks, and you can explain why it works. Separate independent and assisted solves. A correct slow baseline is partial if it misses the required bound. Testing time counts toward the session. If finished early, improve tests and proofs without increasing the solved count.

Attempt record

For each problem, submit its number, minutes spent, independent/assisted/partial/parked status, hints used, code, test results, invariant, and time/space analysis. Keep the same language throughout the timed week.

At 120 minutes record: attempted __; independently solved __; assisted solves __; partial __.

Exit ticket: Which assumption or boundary was hardest? Give one smallest failing example and explain your correction. Counts are evidence of this session, not a quota.